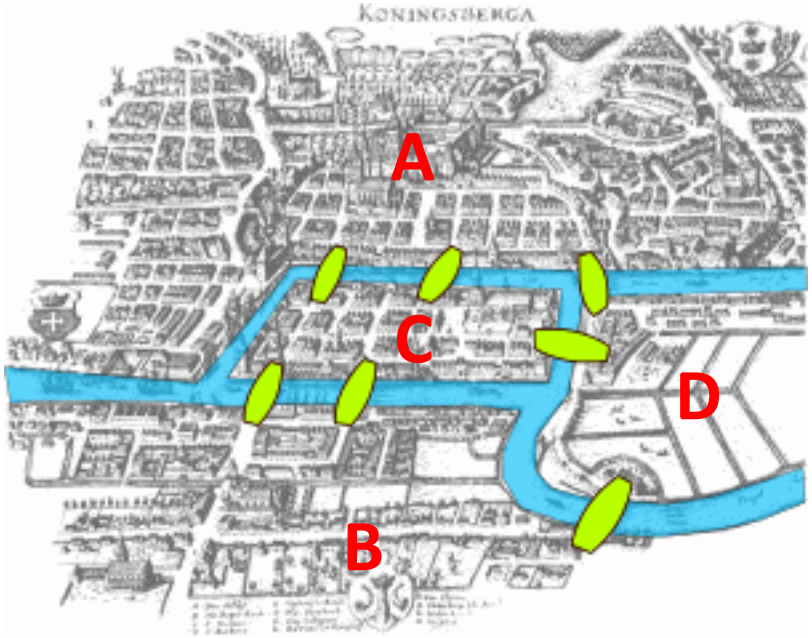
ECE 4502/6502 & CS 6501: Machine Learning on Graphs

Lecture 2. Graph Essentials

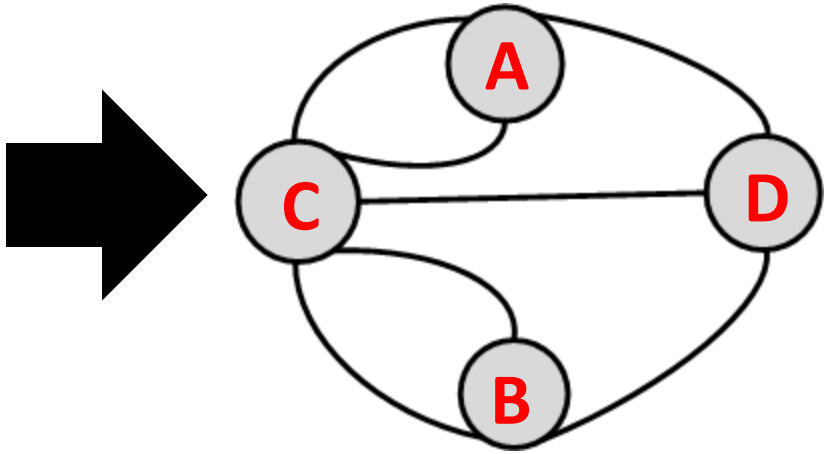
Instructor: Jundong Li
Spring 2025, University of Virginia

Bridges of Konigsberg

- There are **2 islands** and **7 bridges** that connect the islands and the mainland
- Find a path that crosses each bridge exactly once



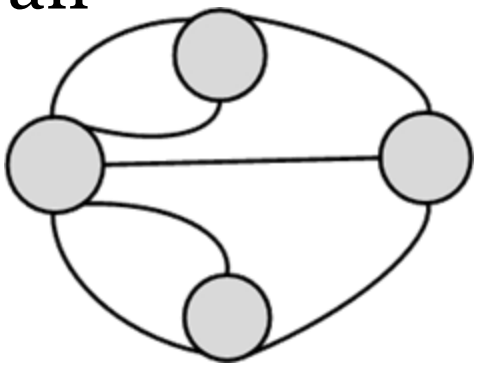
City Map



Graph Representation

Modeling by Graph Theory

- The key to solving this problem is an ingenious graph representation
  - What should be nodes and edges?



- Euler proved that except for the starting and ending point of a walk, one has to enter and leave all other nodes, thus these nodes should have an even number of bridges
- This property does not hold in this problem

Graph Basics

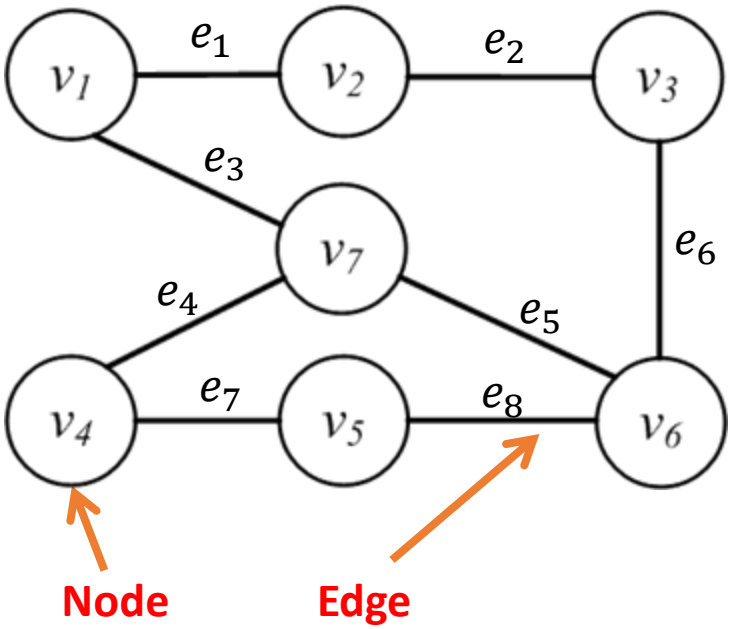
Definition of Graphs

- Graphs/networks (G) are a general data structure to depict the interactions between different entities
  - Entities: nodes/vertices (V)
    $$V = \{v_1, v_2, \ldots, v_n\}$$
  - Interactions: edges (E)
    $$E = \{e_1, e_2, \ldots, e_m\}$$
    $e_8(e_{56})$=<v_5, v_6>

Start node

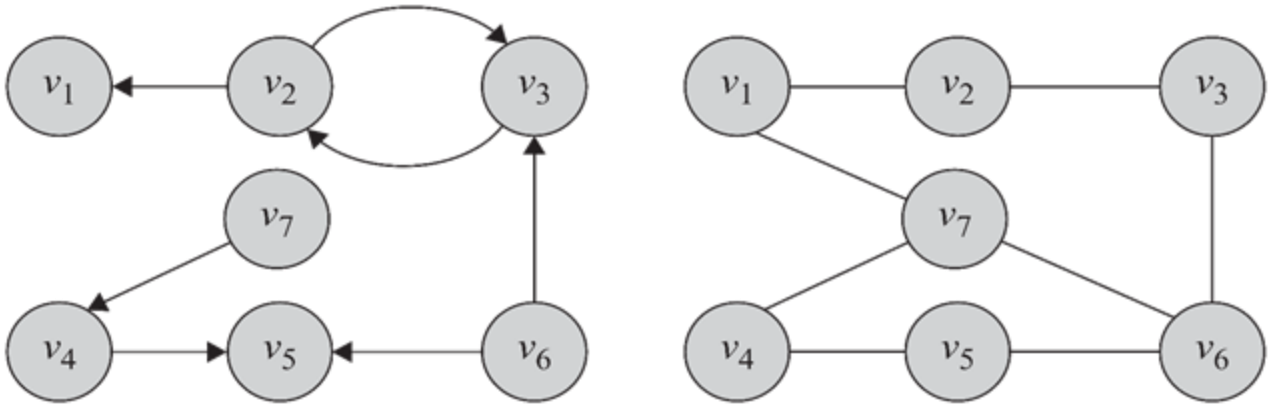
End node

- $G =< V, E >$
  - Number of nodes $|V| = 6$
  - Number of Edges $|E| = 8$



Directed Edges and Directed Graphs

- Edges can have directions. A directed edge is sometimes called an arc



(a) Directed Graph (b) Undirected Graph

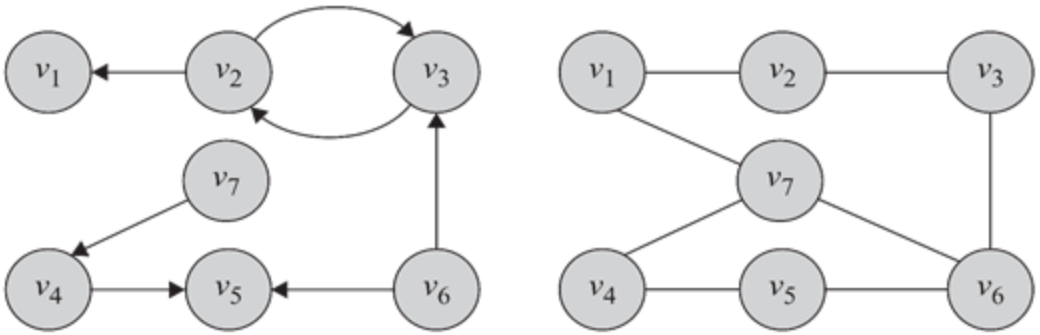
- Directed graph: $< v_2, v_3 > \neq < v_3, v_2 >$
- Undirected graph: $< v_2, v_3 > = < v_3, v_2 >$

Node Degree

- The number of edges connected to one node is the degree of that node
  - Undirected graph: degree of a node i is usually presented using notation d_i
  - Directed graph:
    - *In-degree:* the number of edges **pointing towards** a node d_i^{in}
    - *Out-degree:* the number of edges **pointing away** from a node d_i^{out}

$d_3^{in} = 2$

$d_3^{out} = 1$



(a) Directed Graph

(b) Undirected Graph

$d_7 = 3$

Node Degree

- **Theorem 1.** The summation of degrees in an undirected graph is twice the number of edges

$$\sum_i d_i = 2|E|$$

- **Lemma 1.** The number of nodes with odd degree is even
- **Lemma 2.** In any directed graph, the summation of in-degrees is equal to the summation of out-degrees

$$\sum_i d_i^{out} = \sum_j d_j^{in}$$

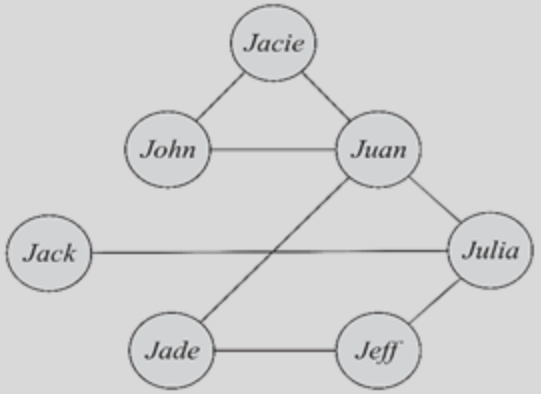
Degree Distribution

- When dealing with very large graphs, how nodes' degrees are distributed is an important concept to analyze and is called ***Degree Distribution***

$\pi(d) = \{d_1, d_2, \ldots, d_n\}$ (Degree sequence)

$p_d = \frac{n_d}{n}$

n_d is the number of nodes with degree d

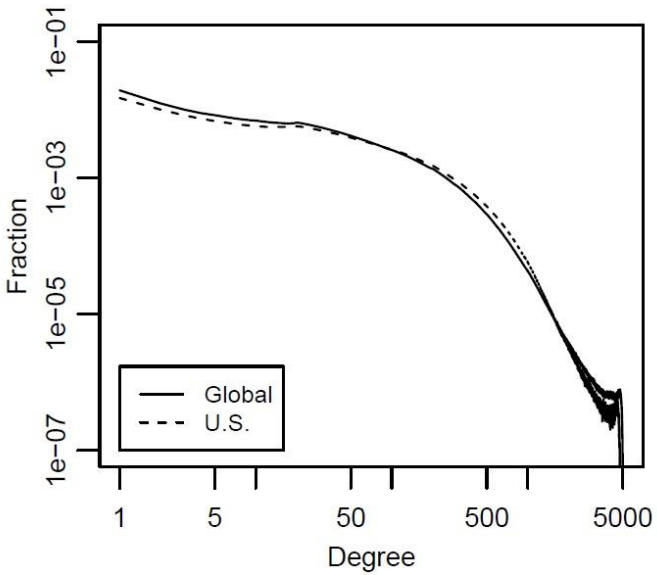


$p_1 = \frac{1}{7}, p_2 = \frac{4}{7}, p_3 = \frac{1}{7}, p_4 = \frac{1}{7}$

Degree Distribution Plot

- The x-axis represents the degree and the y-axis represents the fraction of nodes having that degree
- On social networking sites
  - There exist many users with few connections and there exist a handful of users with very large numbers of friends

(Power-law degree distribution)



Facebook Degree Distribution

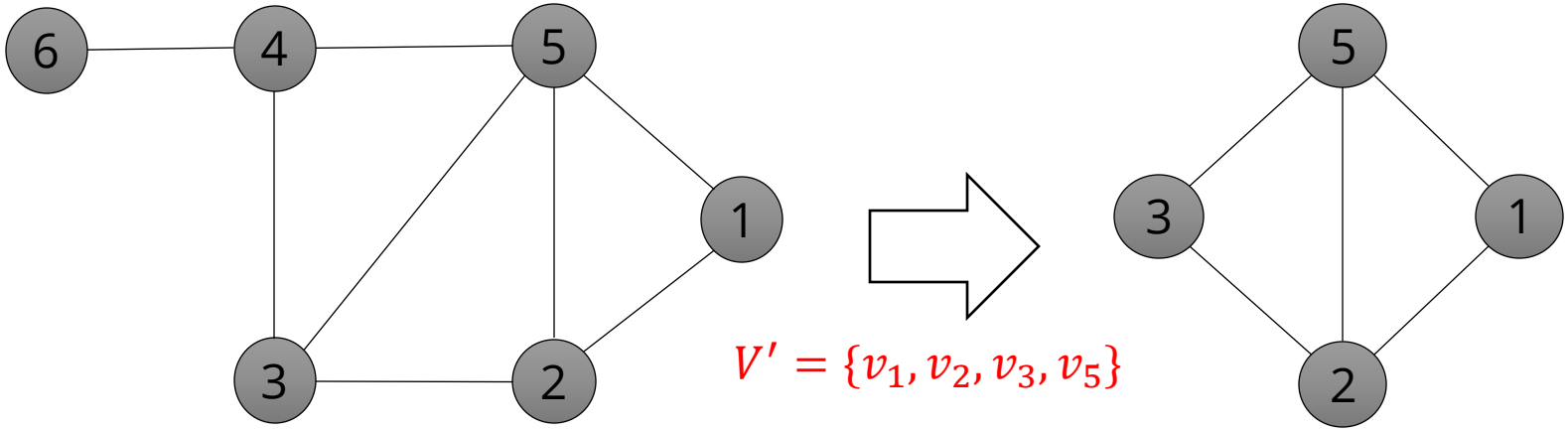
Subgraph

- $G'(V', E')$ is a subgraph of $G(V, E)$ if

$$V' \subseteq V \qquad E' \subseteq (V' \times V') \cap E$$

The edge in between is part of the original graph

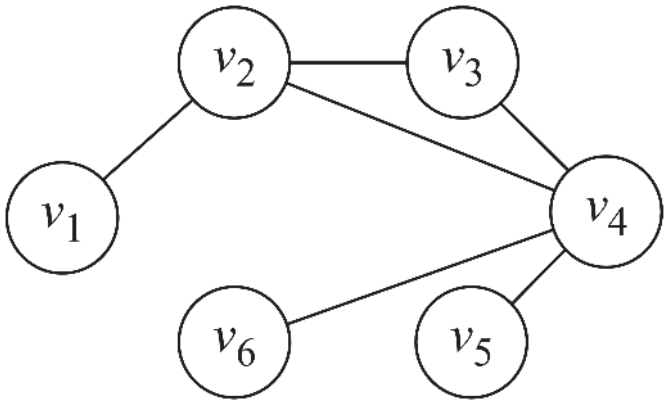
Both end nodes are in node set V'



Graph Representation

Adjacency Matrix

$$A_{ij} = \begin{cases} 1, \text{ if there is an edge between nodes } v_i \text{ and } v_j \\ 0, \text{ otherwise} \end{cases}$$



(a) Graph

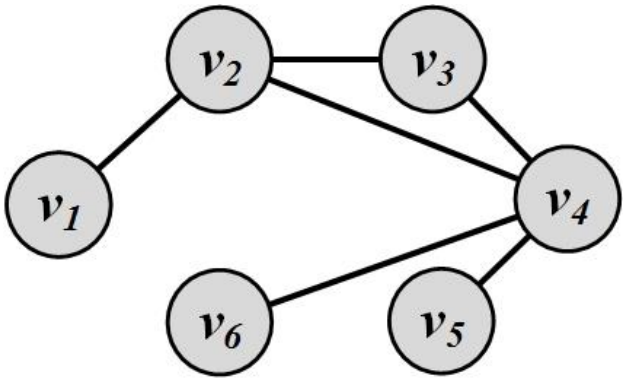
	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	1	0	0	0	0
v_2	1	0	1	1	0	0
v_3	0	1	0	1	0	0
v_4	0	1	1	0	1	1
v_5	0	0	0	1	0	0
v_6	0	0	0	1	0	0

(b) Adjacency Matrix

The adjacency matrix of many real-world graphs are very sparse

Adjacency List

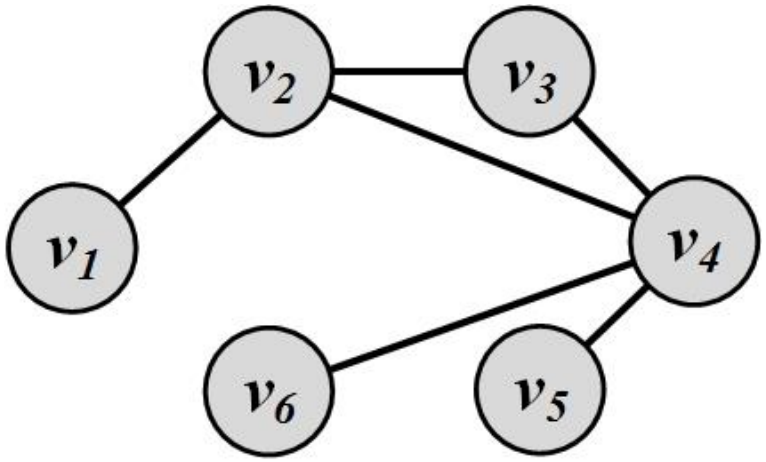
- In an adjacency list for every node, we maintain a list of all the nodes that it is connected to
- The list is usually sorted based on the node order or other preferences



Node	Connected To
v_1	v_2
v_2	v_1, v_3, v_4
v_3	v_2, v_4
v_4	v_2, v_3, v_5, v_6
v_5	v_4
v_6	v_4

Edge List

- In this representation, each element is an edge and is usually represented as (u, v), denoting that node u is connected to node v via an edge

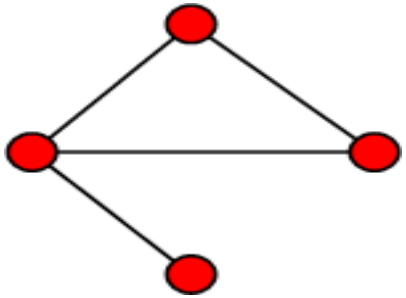


(v_1, v_2)
(v_2, v_3)
(v_2, v_4)
(v_3, v_4)
(v_4, v_5)
(v_4, v_6)

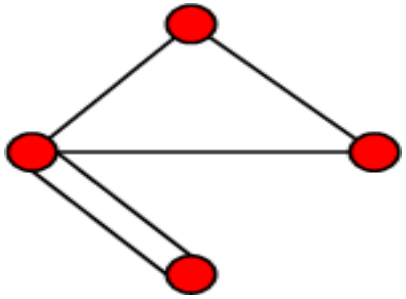
Types of Graphs

Simple Graphs and Multigraphs

- Simple graphs are graphs where only a single edge can be between any pair of nodes
- Multigraphs are graphs where you can have multiple edges between two nodes and loops



Simple graph

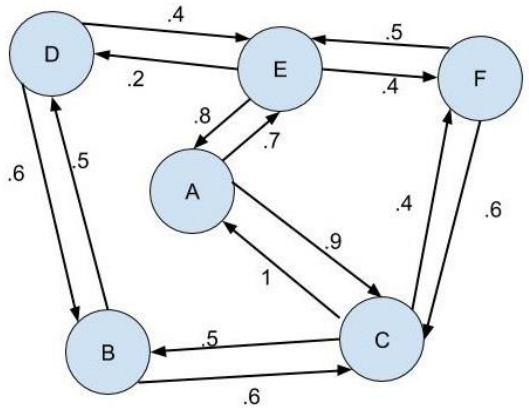


Multi graph

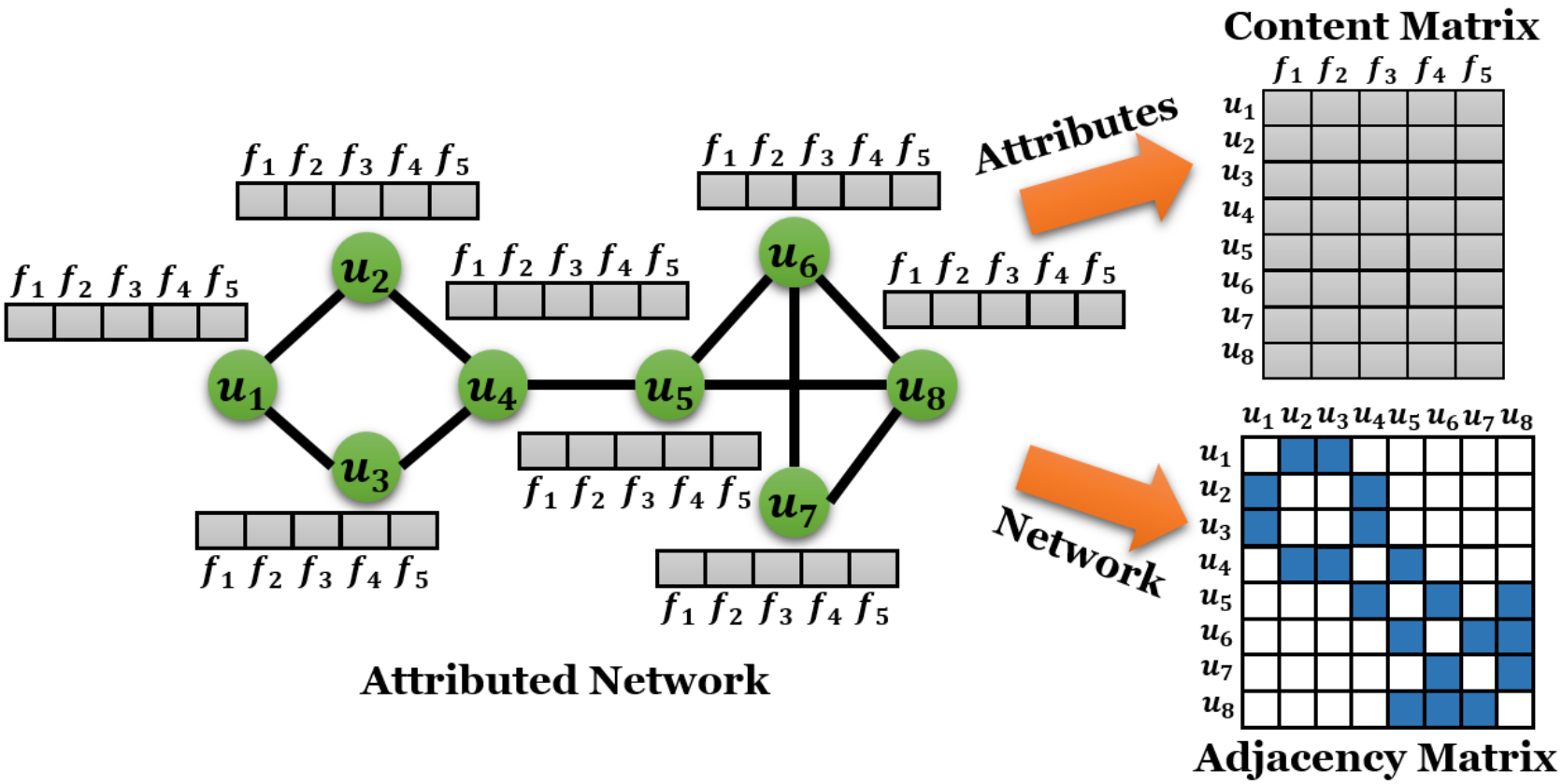
- The adjacency matrix for multigraphs can include numbers larger than one, indicating multiple edges between nodes

Weighted Graph

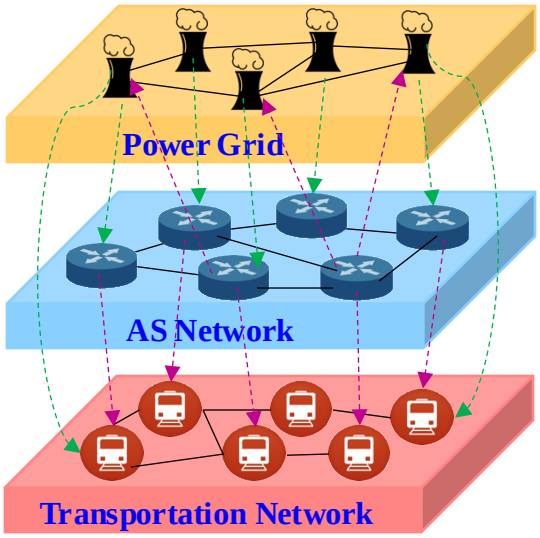
- Weighted graph $\boldsymbol{G(V, E, W)}$
  - Edges are associated with weights
  - Weight of edge e_{ij} is often denoted as w_{ij}
- Example
  - Social strength in social networks

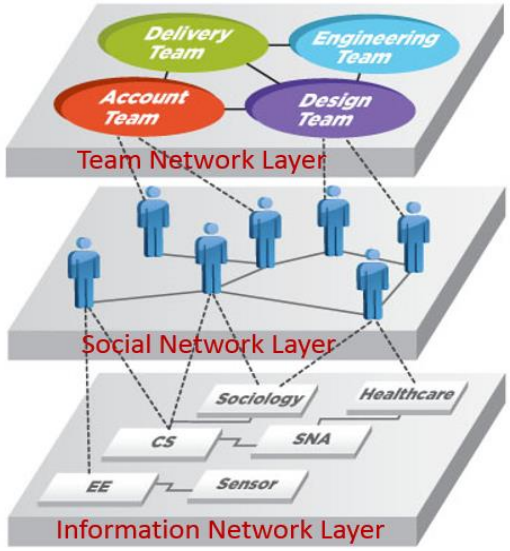


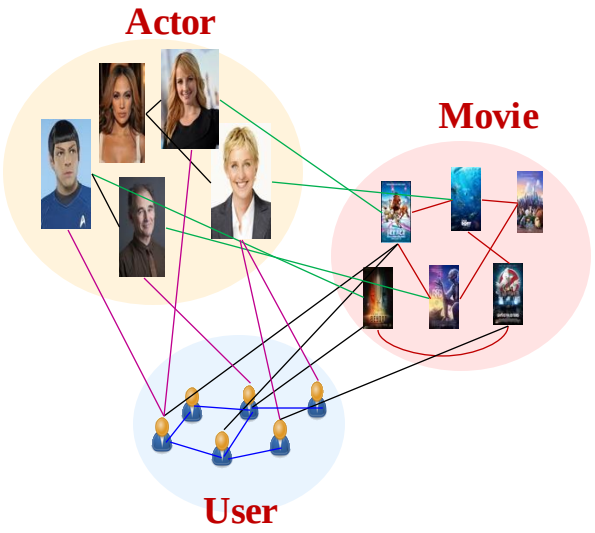
Attributed Networks



Multi-Layered Graphs



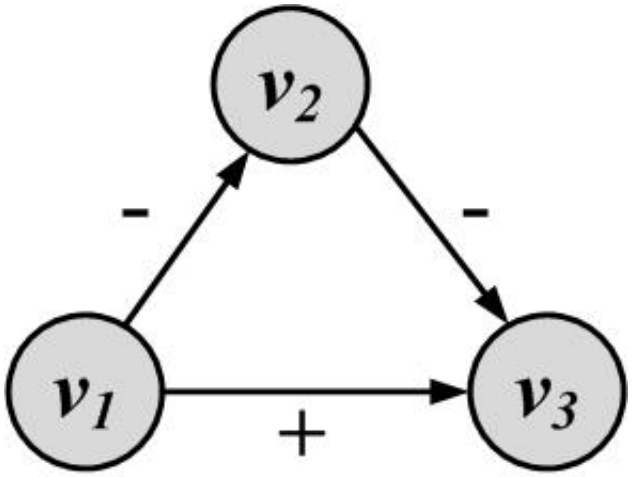




	Infrastructure Networks	Collaboration Platforms	Recommender Systems
Intra-Layer	Power Grid AS Network Transportation Network	Team Network Social Network Information Network	Actor Network Movie Network User Social Network
Cross-Layer	Power Station↔Routers Power Station↔Transportation Routers↔Transportation	Team↔Employees Employee↔Information	Actor↔User User↔Movie Movie↔Actor

Signed Graph

- When weights are binary (0/1, -1/1, +/-) we have a signed graph



- It is used to represent friends or foes
- It is also used to represent social status

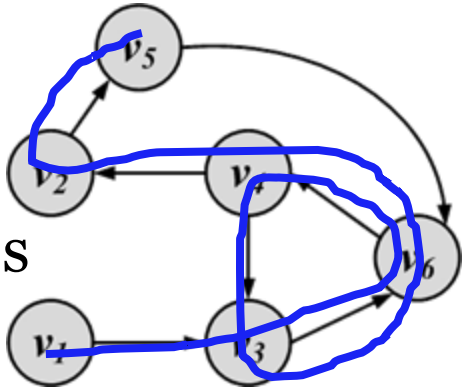
Connectivity in Graphs

Walk

- **Walk**: A walk is a sequence of incident edges visited one after another
  - **Open walk**: A walk does not end where it starts
  - **Closed walk**: A walk returns to where it starts

- Representing a walk:
  - A sequence of edges: $e_1, e_2, \ldots, e_n$
  - A sequence of nodes: $v_1, v_2, \ldots, v_n$

- Length of walk: the number of visited edges

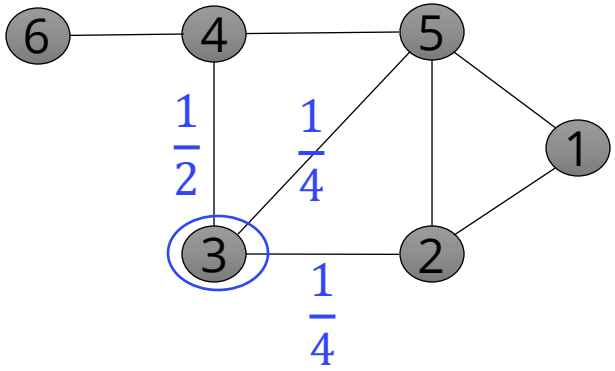


Length of walk = 8

Random Walk

- A walk that in each step the next node is selected **randomly** among the neighbors
  - Directed graph: randomly among **outgoing** neighbors
- The weight of an edge can be used to define the probability of visiting it
- For all edges that start at v_i the following equation holds

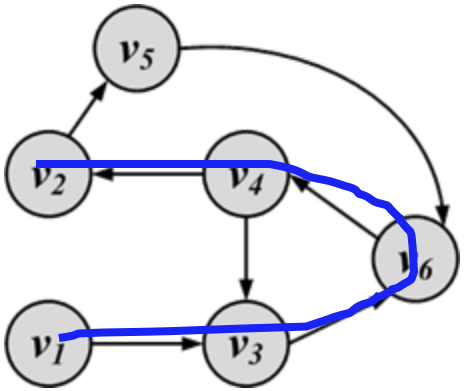
$$\sum_x w_{i,x} = 1, \forall i, j \quad w_{i,j} \geq 0$$



Path

- A walk where **nodes and edges** **are distinct** is called a **path** and a closed path is called a **cycle**

- **The length of a path** or cycle is the number of edges visited in the path or cycle

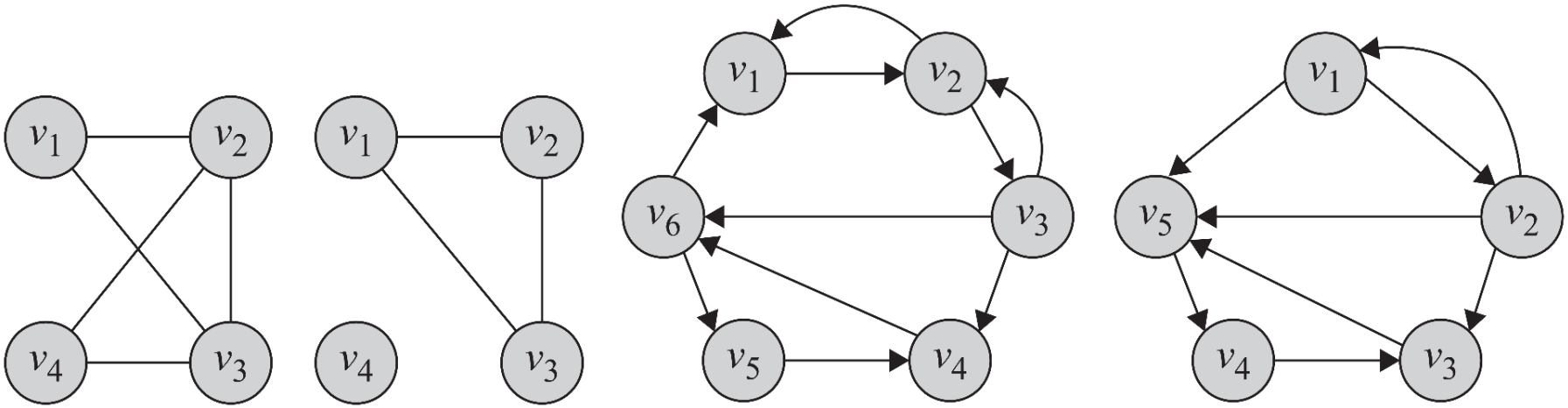


Length of path = 4

Connectivity

- **A node v_i is connected to node v_j** (or reachable from v_j) if it is adjacent to it or there exists a path from v_i to v_j
- **A graph is connected**, if there exists a path between any pair of nodes in it
  - In a directed graph, **a graph is strongly connected** if there exists a directed path between any pair of nodes
  - In a directed graph, **a graph is weakly connected** if there exists a path between any pair of nodes, without following the edge directions
- A graph is **disconnected,** if it not connected

Connectivity: Example



(a) Connected (b) Disconnected (c) Strongly connected (d) Weakly connected

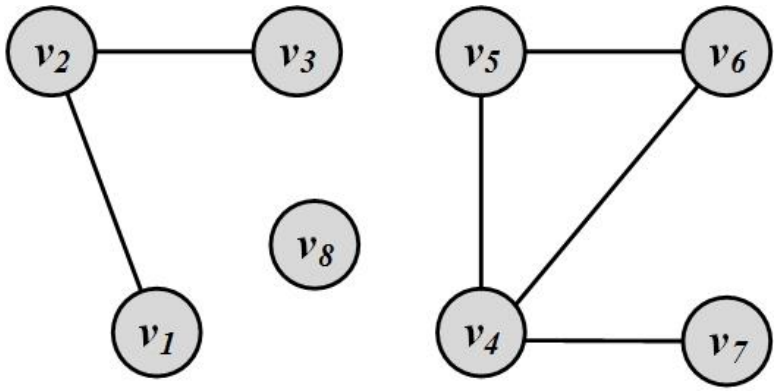
Component

- A **component** in an undirected graph is a connected **subgraph**, i.e., there is a path between every pair of nodes inside the component

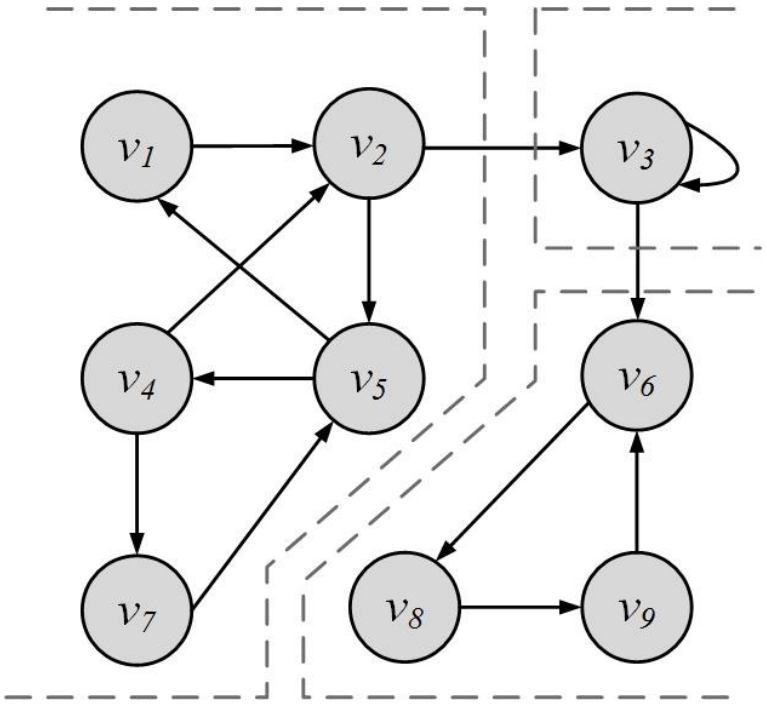
- In directed graphs, we have a **strongly connected** components when there is a path from u to v and one from v to u for every pair of nodes u and v

- The component is **weakly connected** if replacing directed edges with undirected edges results in a connected component

Component Examples:



3 components



3 Strongly-connected components

Shortest Path

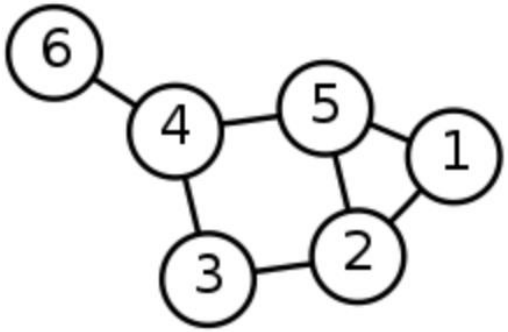
- **Shortest Path** is the path between two nodes that has the shortest length
  - We denote the length of the shortest path between nodes v_i and v_j as $l_{i,j}$

- The concept of the neighborhood of a node can be generalized using shortest paths. An **n-hop neighborhood** of a node is the set of nodes that are within n hops distance from the node

Diameter

The diameter of a graph is the length of the longest shortest path between any pair of nodes between any pairs of nodes in the graph

$$diameter_G = \max_{(v_i, v_j) \in V \times V} l_{i,j}$$

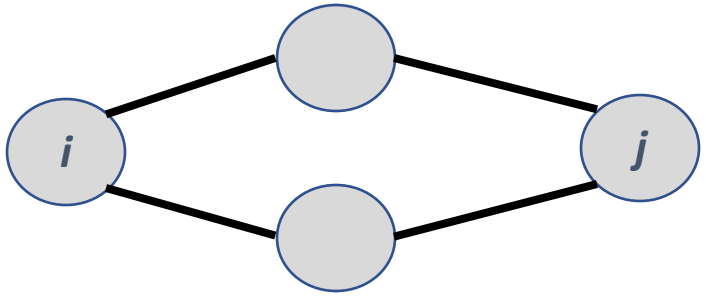


How large is the web graph?

Adjacency Matrix and Connectivity

- Consider the following adjacency matrix

$$A = \begin{bmatrix} A_{11} & A_{12} & A_{13} & \dots & A_{1n} \\ A_{21} & A_{22} & A_{23} & \dots & A_{2n} \\ \dots & \dots & \dots & \dots & \dots \\ A_{d1} & A_{d2} & A_{d3} & \dots & A_{dn} \end{bmatrix}$$

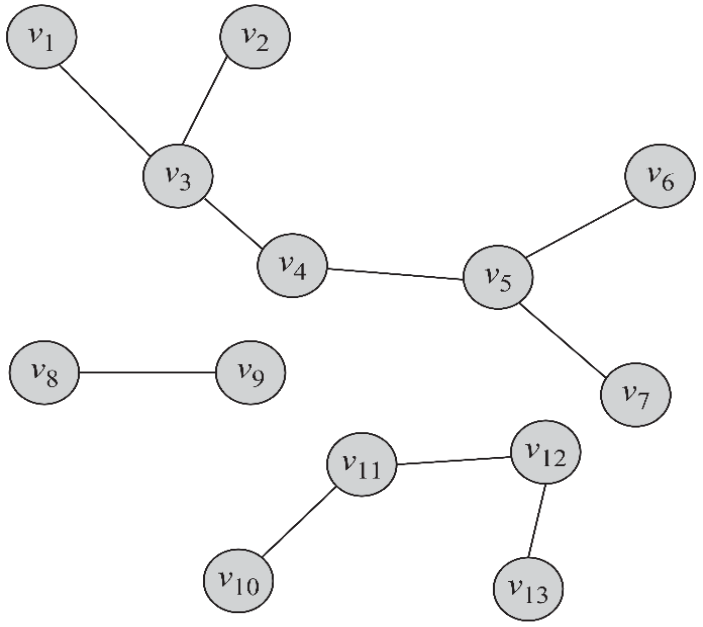


- Number of common neighbors between node i and node j $\sum_k A_{ik}A_{jk} = A_i \cdot A_j$
- That's element of $[ij]$ of matrix $A \times A^T = A^2$
- Common neighbors are walks of length 2

Special Graphs

Trees and Forests

- **Trees** are special cases of undirected graphs
- A tree is a graph structure that has no cycle in it
- In a tree, there is exactly one path between any pair of nodes
- In a tree: $|V| = |E| + 1$
- A set of disconnected trees is called a **forest**

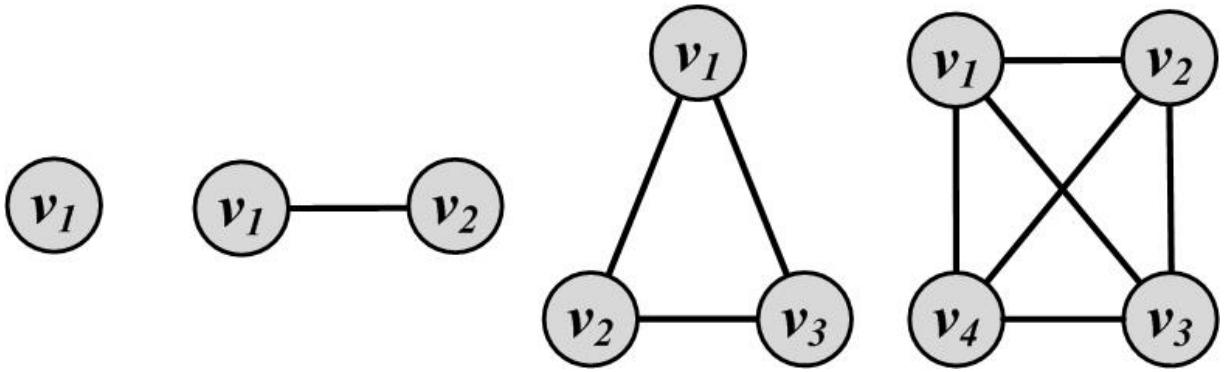


A forest containing 3 trees

Complete Graphs

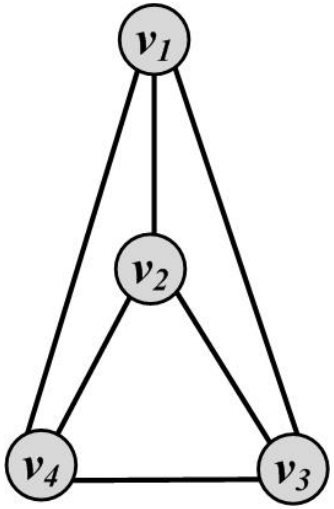
- A complete graph is a graph where for a set of nodes V, all possible edges exist in the graph
- In a complete graph, any pair of nodes are connected via an edge

$$|E| = \binom{|V|}{2}$$

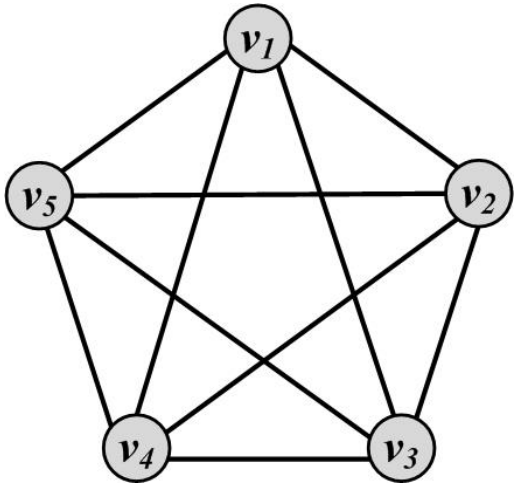


Planar Graphs

- A graph that can be drawn in such a way that no two edges cross each other (other than the endpoints) is called planar



Planar Graph

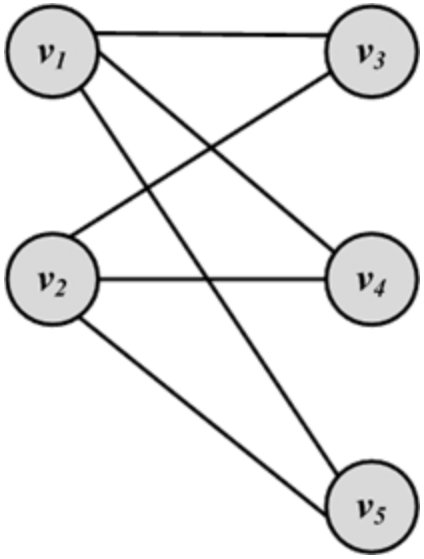


Non-planar Graph

Bipartite Graphs

A bipartite graph $G(V, E)$ is a graph where the node set can be partitioned into two sets such that, for all edges, one end-point is in one set and the other end-point is in the other set

$$\begin{cases} V = V_L \cup V_R, \\ V_L \cap V_R = \emptyset, \\ E \subset V_L \times V_R \end{cases}$$

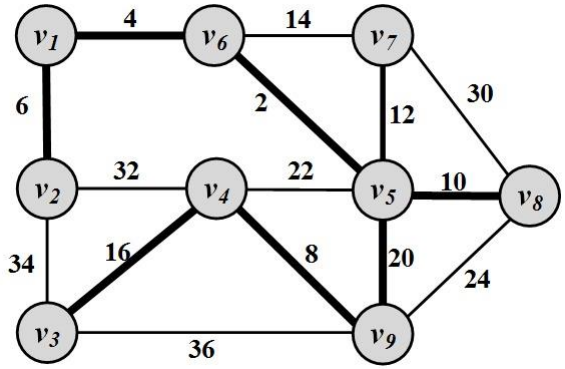


Special Subgraphs

Spanning Trees

- For any connected graph, the spanning tree is a subgraph and a tree that includes all the nodes
- There may exist multiple spanning trees for a graph
- In a weighted graph, the weight of a spanning tree is the summation of the edge weights in the tree
- Among the many spanning trees found for a weighted graph, the one with the minimum weight is called the

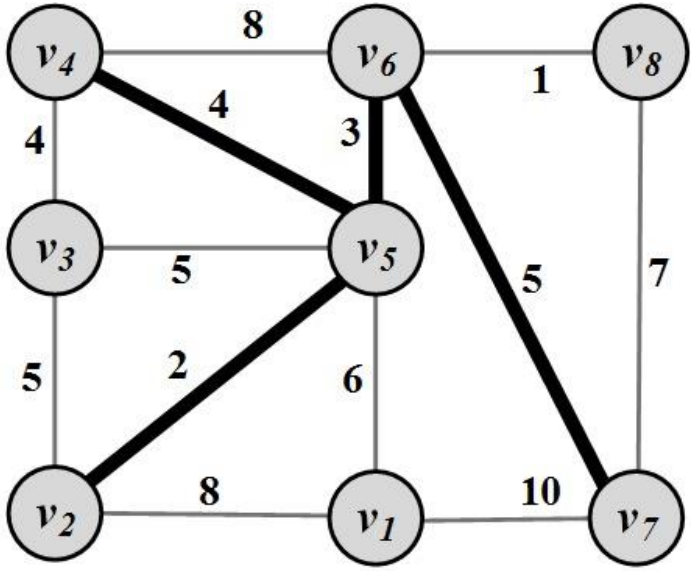
minimum spanning tree (MST)



Steiner Trees

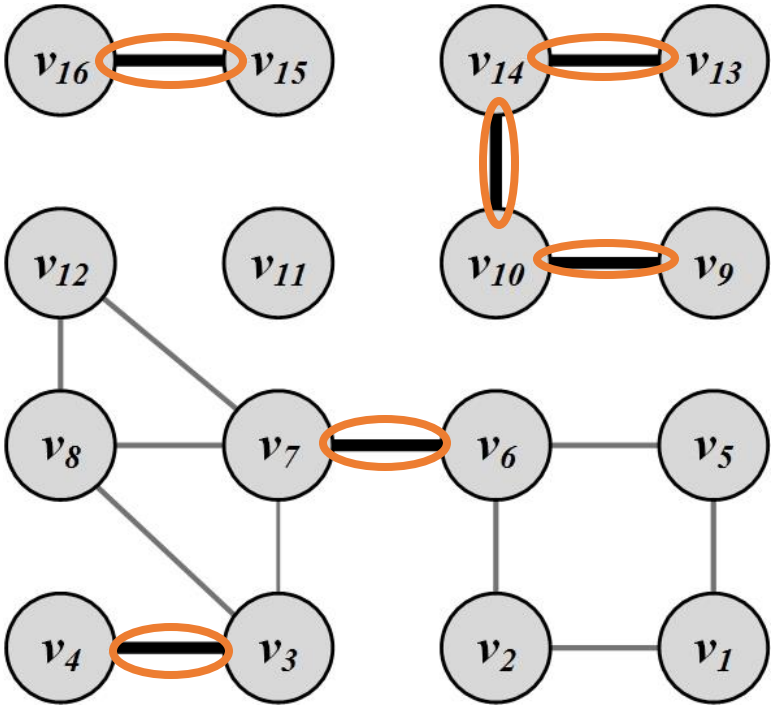
Given a weighted graph $G(V, E, W)$ and a ***subset*** of nodes $V' \subseteq V$ (terminal nodes), the Steiner tree problem aims to find a tree such that it spans all the V' nodes and the **weight of this tree is minimized**

$$V' = \{v_2, v_4, v_5, v_6, v_7\}$$



Bridges (cut-edges)

- Bridges are edges whose removal will increase the number of connected components



Graph Algorithms

Graph/Network Traversal Algorithms

Graph/Tree Traversal

- Goal:
  - Visit all nodes in the graph by starting from a given node v
- Key idea:
  - Expanding the neighborhood of node v until all nodes are visited
- The traversal technique guarantees that
  - All users are visited; and
  - No user is visited more than once
- Two main techniques:
  - Depth-First Search (DFS)
  - Breadth-First Search (BFS)

Depth-First Search (DFS)

- Key idea:
  - starts from a node v_i, selects one of its neighbors v_j from $N(v_i)$ and performs Depth-First Search on v_j before visiting other neighbors in $N(v_i)$
- The algorithm can be used both for trees and graphs
- The algorithm can be implemented using a **stack structure** **(First in Last out)**

DFS Algorithm

Algorithm 2.2 Depth-First Search (DFS)

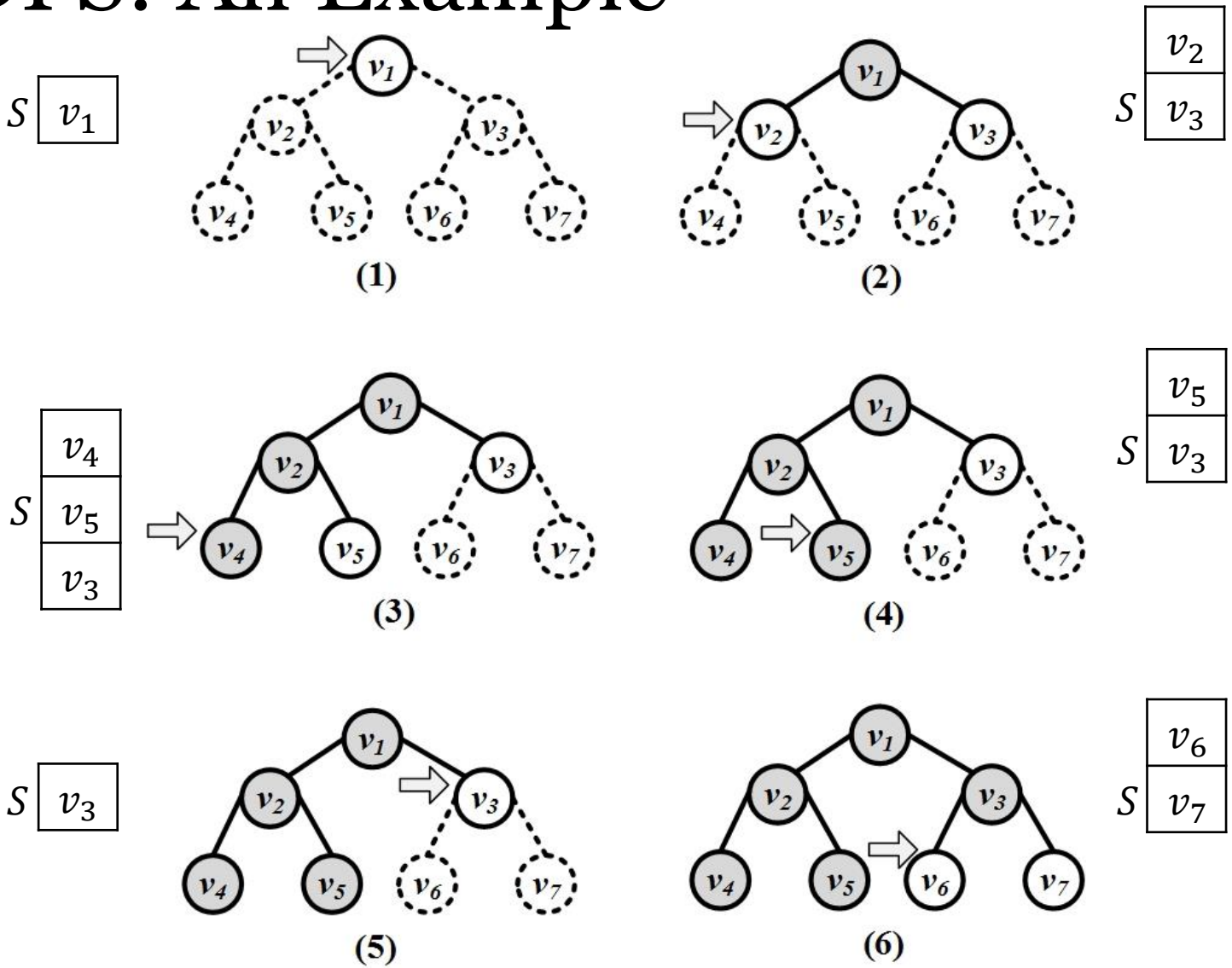
Require: Initial node v, graph/tree $G(V, E)$, stack S

1: **return** An ordering on how nodes in G are visited
2: Push v into S;
3: *visitOrder* = 0;
4: **while** S not empty **do**
5: *node* = pop from S;
6: **if** *node* not visited **then**
7: *visitOrder* = *visitOrder* +1;
8: Mark *node* as visited with order *visitOrder*; `//or print` *node*
9: Push all neighbors/children of *node* into S;
10: **end if**
11: **end while**
12: Return all nodes with their visit order.

Key Elements:

- Stack
- Node visit tracker

DFS: An Example



Breadth-First Search (BFS)

- Key idea:
  - starts from a node and visits all its immediate neighbors first, and then moves to the second level by traversing their neighbors
- The algorithm can be used both for trees and graphs
- The algorithm can be implemented using a **queue structure** **(First in First out)**

BFS Algorithm

Algorithm 2.3 Breadth-First Search (BFS)

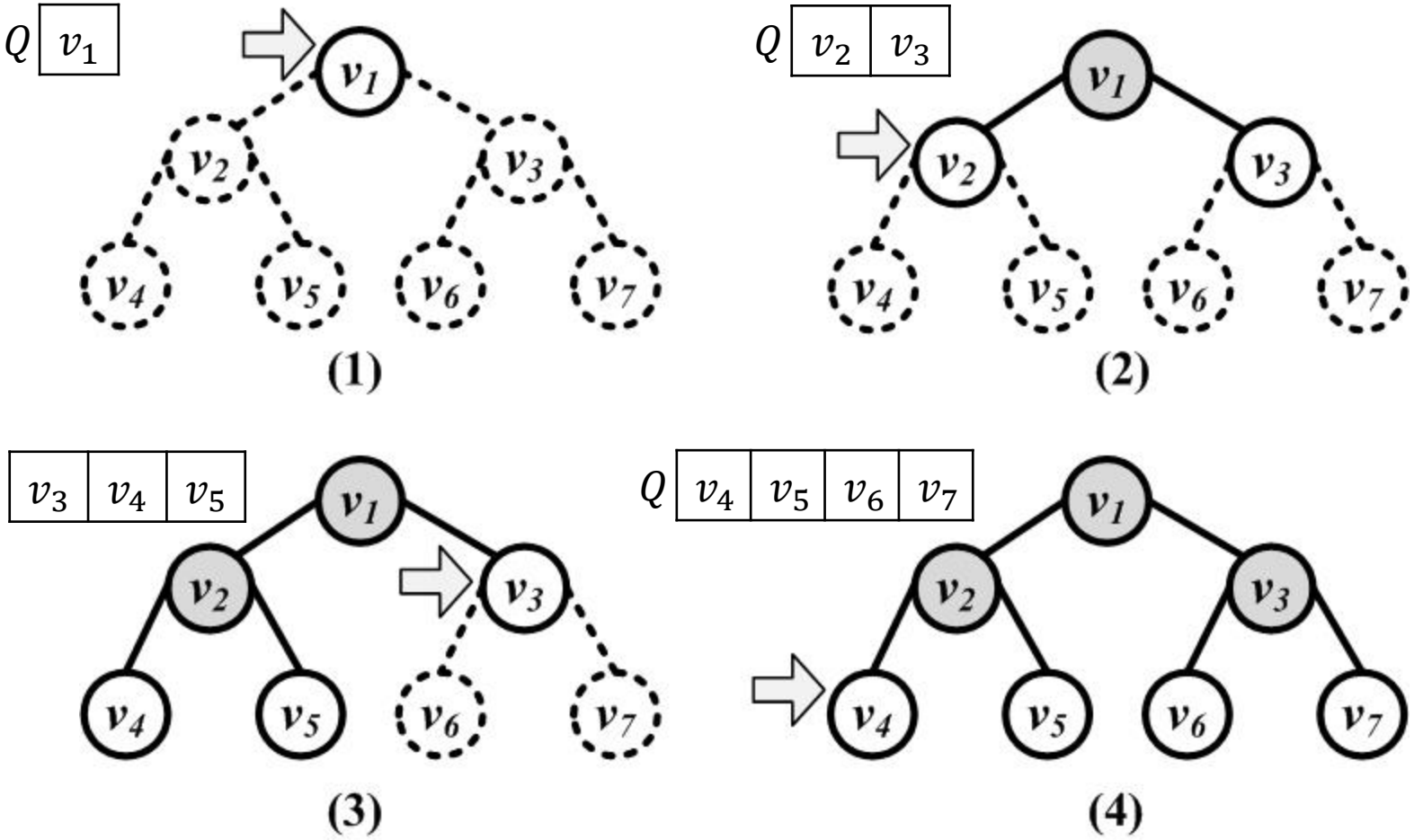
Require: Initial node v, graph/tree $G(V, E)$, queue Q

1: **return** An ordering on how nodes are visited
2: Enqueue v into queue Q;
3: *visitOrder* = 0;
4: **while** Q not empty **do**
5: *node* = dequeue from Q;
6: **if** *node* not visited **then**
7: *visitOrder* = *visitOrder* +1;
8: Mark *node* as visited with order *visitOrder*; `//or print` *node*
9: Enqueue all neighbors/children of *node* into Q;
10: **end if**
11: **end while**

Key Elements:

- Queue
- Node visit tracker

Breadth-First Search (BFS)



Finding Shortest Paths

Shortest Path

- When a graph is connected, there is a chance that multiple paths exist between any pair of nodes
- In many scenarios, we want the shortest path between two nodes in a graph (How fast can I disseminate information on social media?)
- **Dijkstra's Algorithm**
  - Designed for weighted graphs with non-negative edges
  - It finds shortest paths that start from a provided node *s* to all other nodes
  - It finds both shortest paths and their respective lengths

Dijkstra's Algorithm

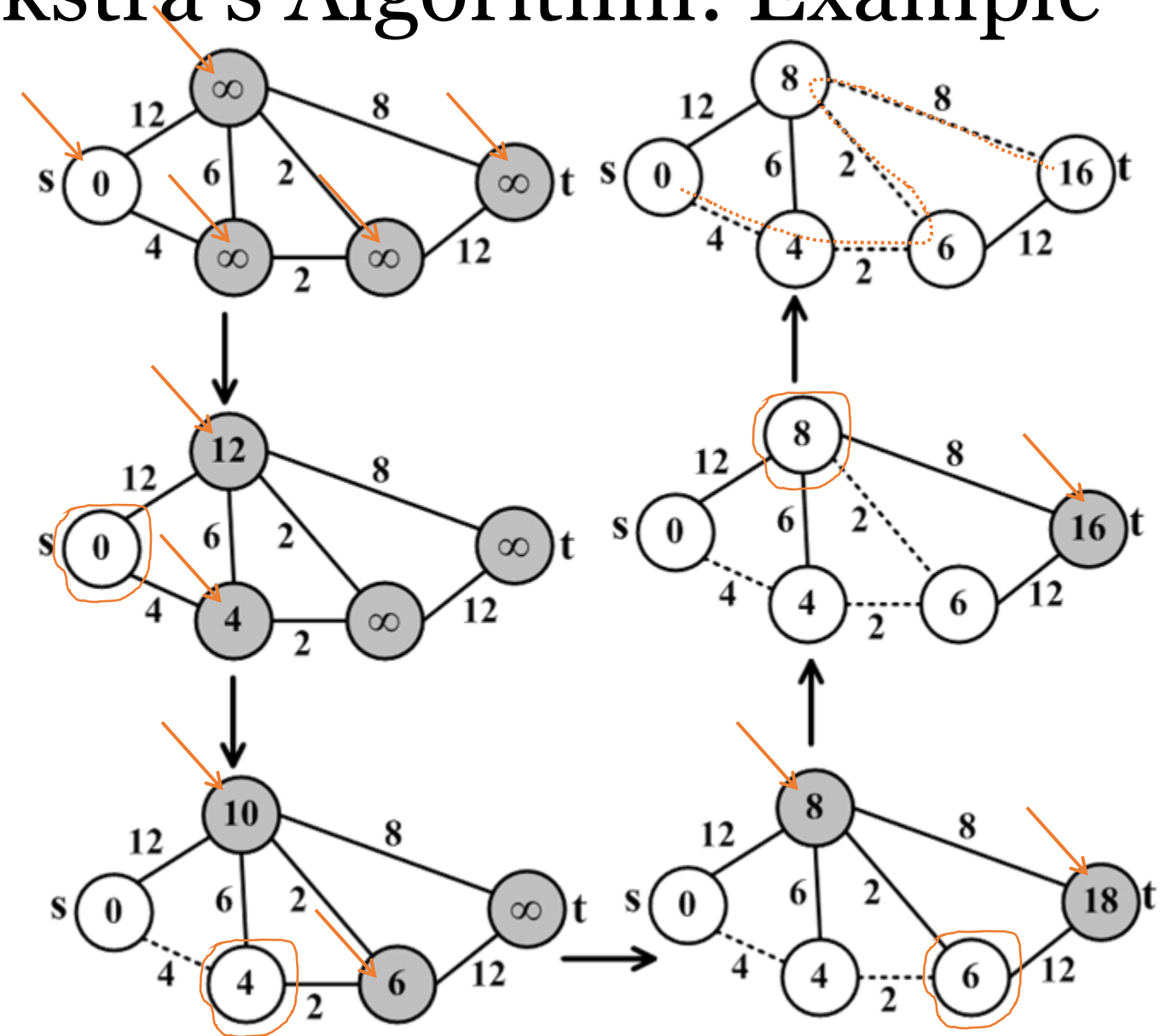
Key Elements:
- Node distance tracker
- Node visit tracker
- Current node pointer

1. Initiation:
   - Assign zero to the source node and infinity to all other nodes
   - Mark all nodes as unvisited
   - Set the source node as current
2. For the current node, consider all of its unvisited neighbors and calculate their *tentative* distances
   - If tentative distance is smaller than neighbor's distance, then Neighbor's distance = tentative distance
3. After considering all of the neighbors of the current node, mark the current node as visited and remove it from the *unvisited set*
4. If the destination node has been marked visited or if the smallest tentative distance among the nodes in the *unvisited set* is infinity, then stop
5. Set the unvisited node marked with the smallest tentative distance as the next "current node" and go to step 2

Tentative distance = current distance + edge weight

A visited node will never be checked again and its distance recorded now is final and minimal

Dijkstra's Algorithm: Example



Dijkstra's Algorithm: Notes

- Dijkstra's algorithm is source-dependent
  - Finds the shortest paths between the source node and all other nodes
- To generate all-pair shortest paths
  - We can run Dijsktra's algorithm n times, or
  - Use other algorithms such as Floyd-Warshall algorithm
- If we want to compute the shortest path from source v to destination d
  - We can stop the algorithm once the shortest path to the destination node has been determined

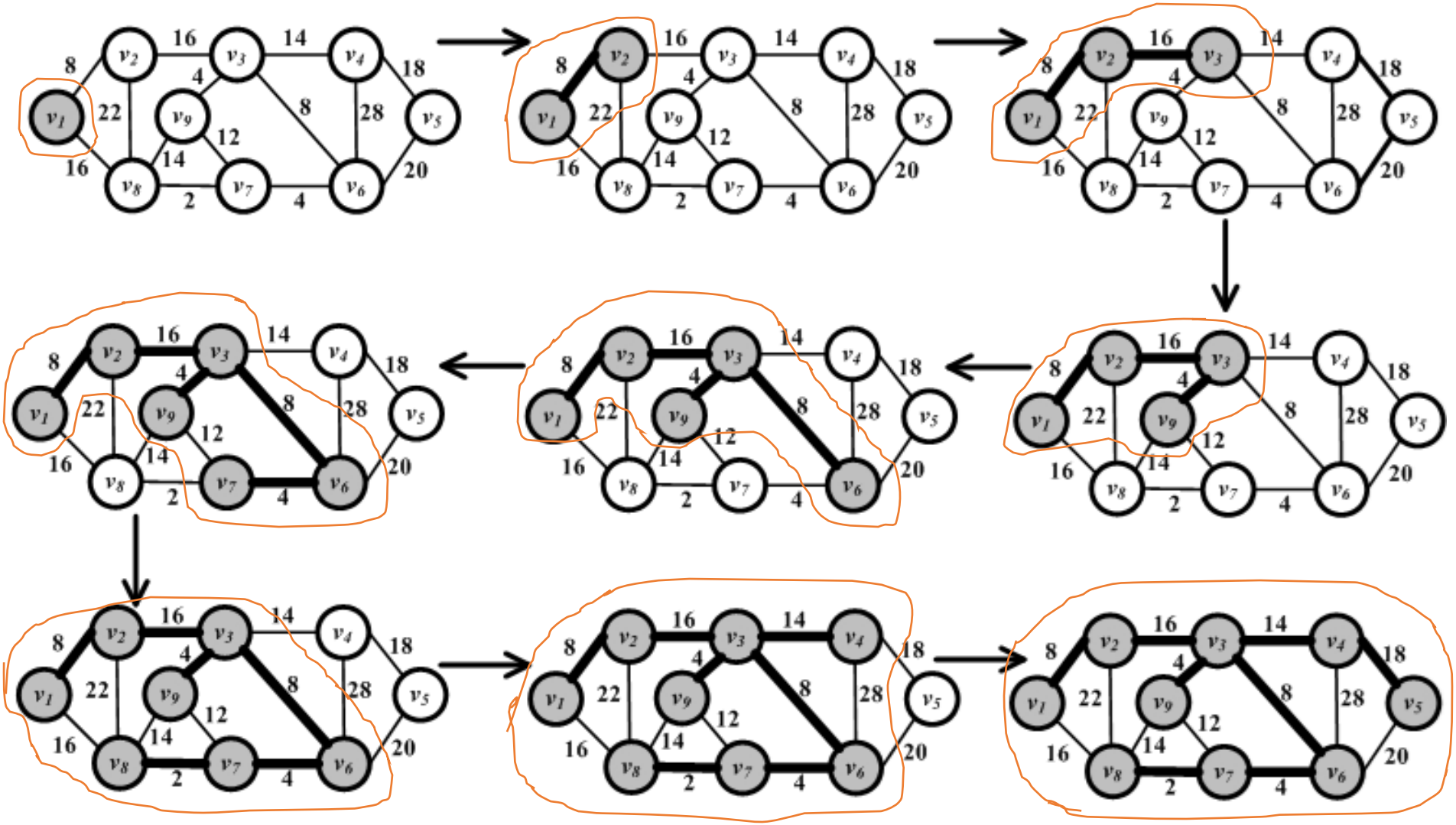
Finding Minimum Spanning Tree

Prim's Algorithm

Finds MST in a weighted graph

1. Selecting a random node and add it to the MST
2. Grows the spanning tree by selecting edges which have one endpoint in the existing spanning tree and one endpoint among the nodes that are not selected yet. Among the possible edges, the one with the minimum weight is added to the set (along with its end-point).
3. This process is iterated until the graph is fully spanned

Prim's Algorithm: Example



Questions?